PRESENTASI TUGAS KELOMPOK

Analisis & Implementasi Perbandingan

Layered vs Non-Layered Web Application Architecture

Studi Kasus: Customer Management System (PHP Native, REST API JSON, & MySQL)

Aplikasi 1: Non-Layered

- Monolitik 1 File (index.php)
- HTML + Validasi + SQL Menyatu

Aplikasi 2: 3-Layer Architecture

- Presentation + Business Logic + Data
- Komunikasi REST API & JSON

Pengujian & Evaluasi

- 8 Skenario Uji CRUD & Validasi
- Analisis 10 Aspek & 5 Pertanyaan

Mata Kuliah Pemrograman Web • Universitas Pendidikan Indonesia

Latar Belakang & Konsep Arsitektur Web

Mengapa Pemisahan Layer Penting dalam Pengembangan Aplikasi Web?

Pendekatan 1: Non-Layered Architecture

- **Definisi:**
  Aplikasi dikembangkan tanpa pemisahan layer tegas (Monolitik).
- **Struktur Kode:**
  HTML, validasi bisnis, dan query SQL menyatu dalam satu file PHP.
- **Cara Kerja:**
  Submit form POST/GET -> PHP proses & query SQL -> Render HTML.
- **Kelebihan:**
  Sangat cepat dibuat untuk aplikasi berskala kecil / sederhana.
- **Kelemahan:**
  Sulit dipelihara, diuji, dan dikembangkan saat fitur bertambah.

Pendekatan 2: 3-Layer Web Architecture

- **Presentation Layer:**
  Menangani tampilan & interaksi pengguna (HTML, CSS, JS Fetch API).
- **Business Logic Layer:**
  Menangani aturan bisnis, validasi data, & endpoint REST API JSON.
- **Data Layer:**
  Menangani koneksi database & query SQL (CustomerRepository & PDO).
- **Komunikasi:**
  Frontend dan Backend berkomunikasi menggunakan REST API & JSON.
- **Keunggulan Utama:**
  Separation of Concerns, Maintainability, Scalability, Security, & Reusability.

Studi Kasus: Customer Management System

Spesifikasi 6 Fitur Wajib dan Desain Tabel Database MySQL (customer_management.sql)

6 Fitur Wajib Aplikasi

1. Tambah Pelanggan

Memasukkan data pelanggan baru (nama, email, telepon, alamat).

2. Tampil Pelanggan

Menampilkan seluruh daftar data pelanggan perusahaan.

3. Edit Pelanggan

Mengubah informasi pelanggan yang sudah terdaftar.

4. Hapus Pelanggan

Menghapus data pelanggan dari database.

5. Pencarian Pelanggan

Mencari pelanggan secara dinamis berdasarkan Nama atau Email.

6. Validasi Data

Memeriksa kelengkapan seluruh field dan keunikan alamat email.

Skema SQL: Tabel `pelanggan` (MySQL)

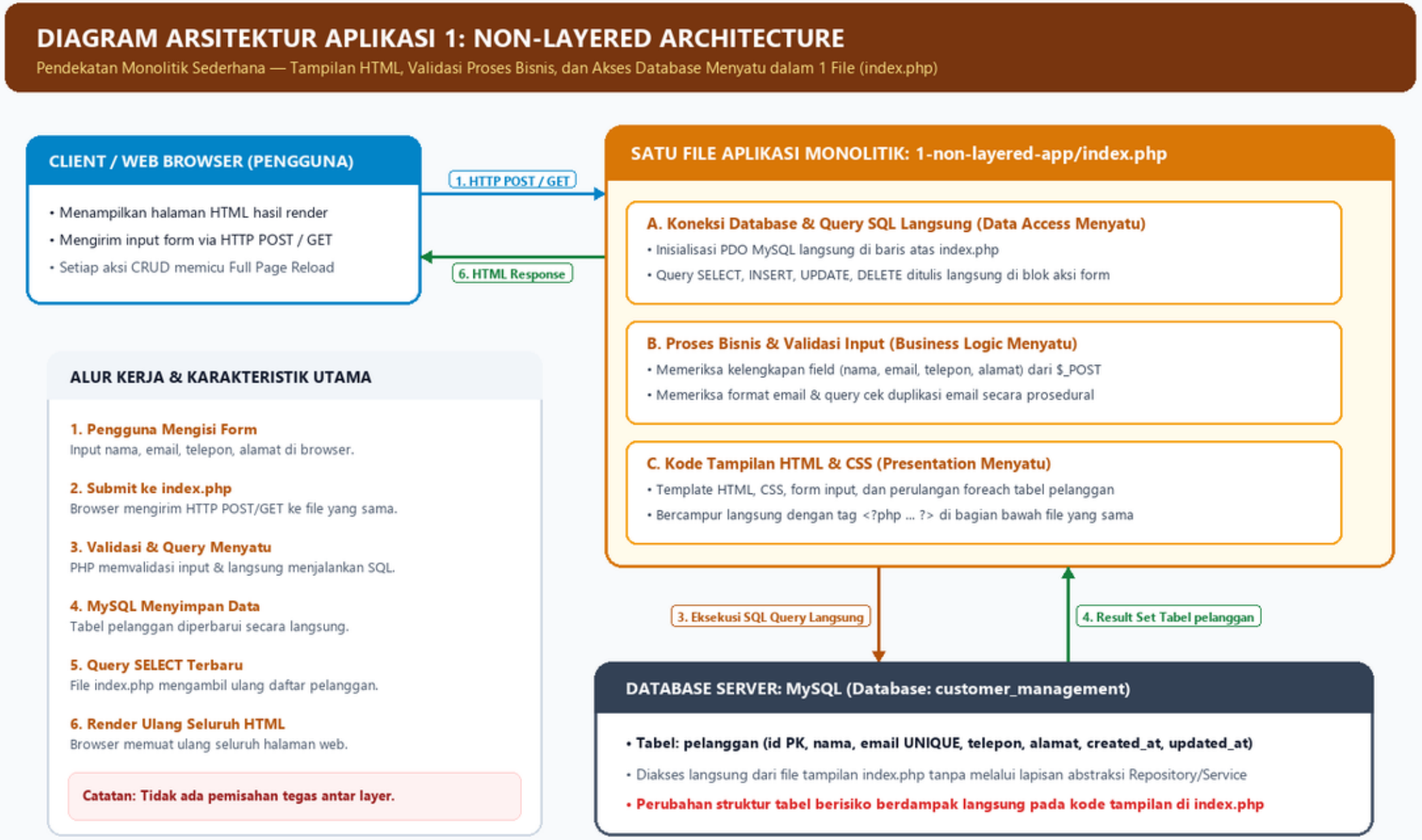
```
CREATE DATABASE IF NOT EXISTS customer_management;
USE customer_management;

CREATE TABLE pelanggan (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nama VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL,
  telepon VARCHAR(30) NOT NULL,
  alamat TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
             ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uk_pelanggan_email (email),
  INDEX idx_pelanggan_nama (nama)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Constraint UNIQUE memastikan integritas
-- keunikan email pada level Database disamping
-- validasi pada Business Logic Layer.
```

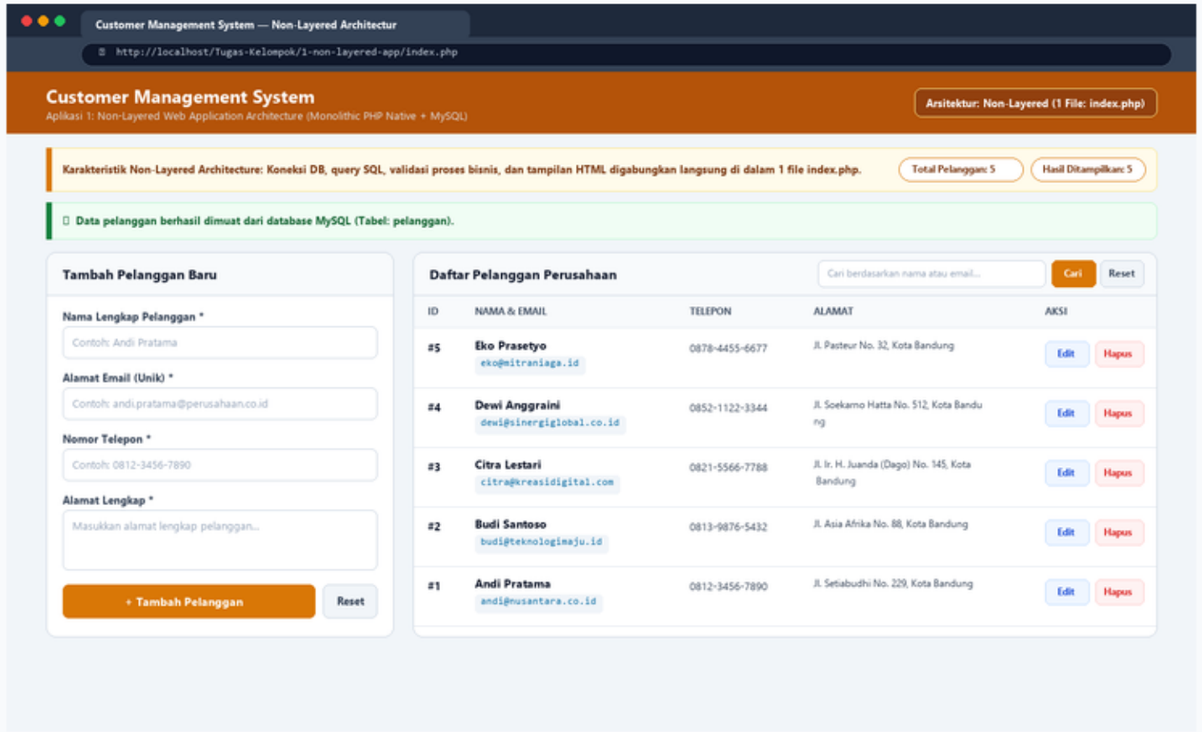
Arsitektur Aplikasi 1: Non-Layered Architecture

Seluruh Tampilan HTML, Proses Bisnis/Validasi, dan Query SQL Berada dalam Satu File (index.php)

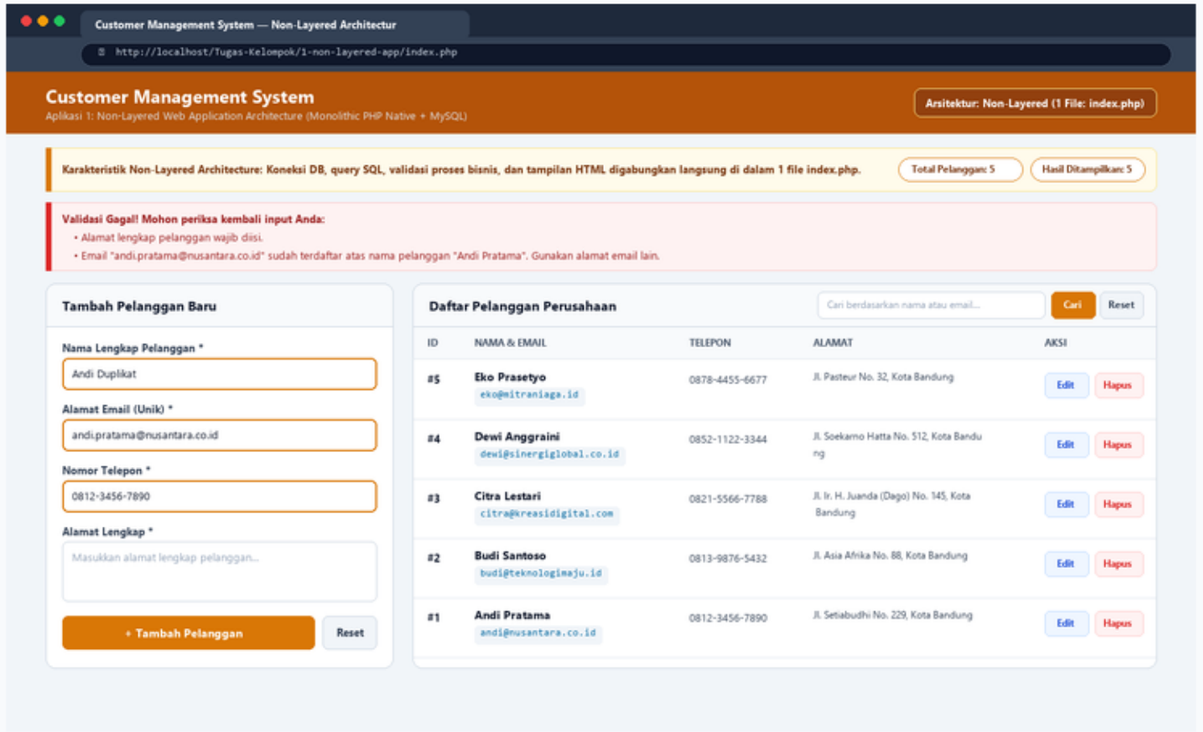


Implementasi & Tampilan Aplikasi 1 (Non-Layered)

Tampilan Utama CRUD serta Validasi Kelengkapan & Keunikan Email pada 1-non-layered-app/index.php



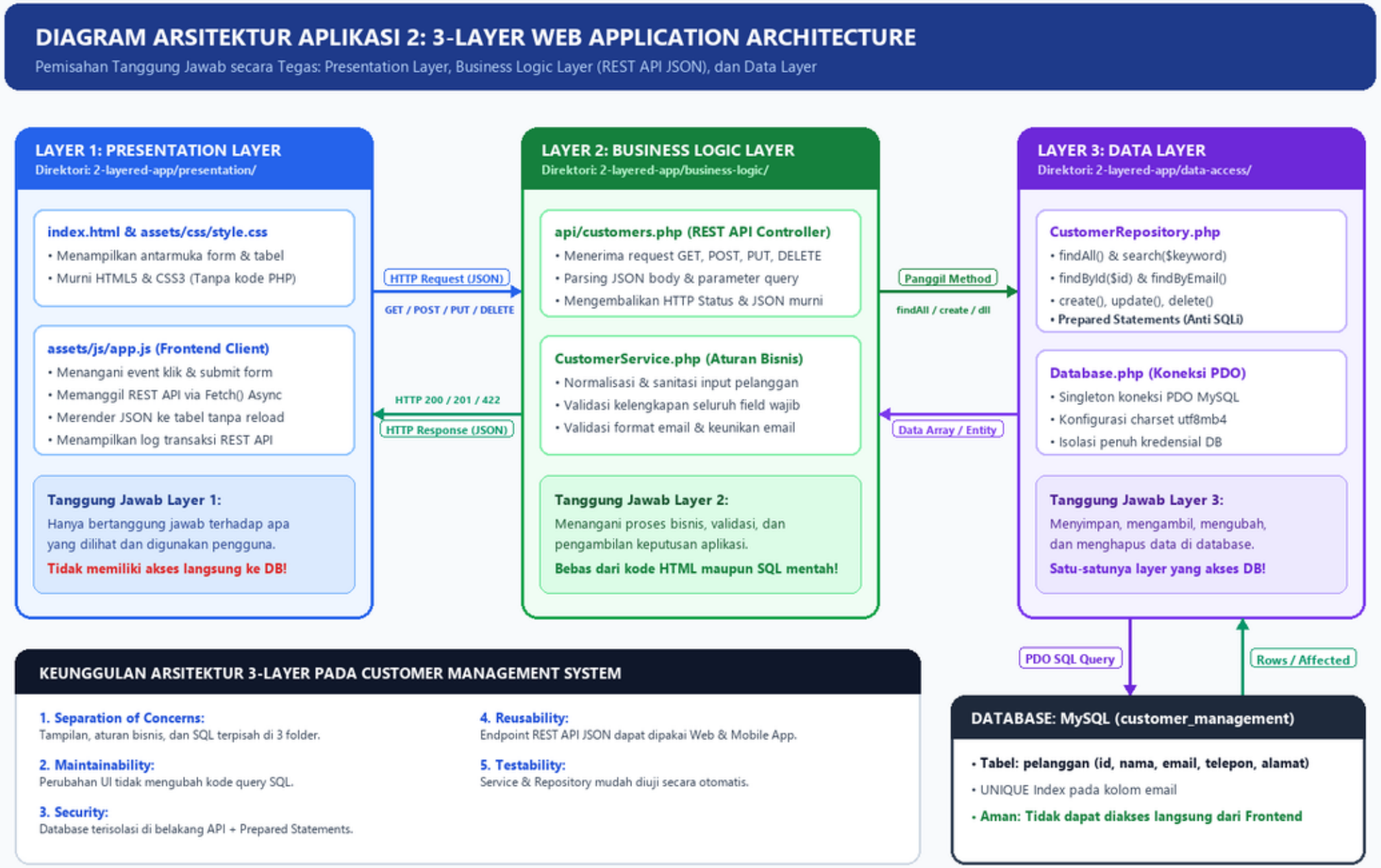
Tampilan Daftar & Tambah Pelanggan (index.php)



Tampilan Validasi Kelengkapan & Email Duplikat

Arsitektur Aplikasi 2: 3-Layer Architecture

Pemisahan Tegas Presentation Layer, Business Logic Layer (REST API JSON), dan Data Layer



Struktur Modul & Endpoint REST API Aplikasi 2

Organisasi File per Layer dan Kontrak Pertukaran Data JSON

Struktur Direktori 2-layered-app/

```
2-layered-app/
├── presentation/                [LAYER 1: FRONTEND]
│   ├── index.html               (Struktur UI HTML5)
│   └── assets/
│       ├── css/style.css        (Styling UI)
│       └── js/app.js            (Fetch REST API JSON)
│
├── business-logic/              [LAYER 2: BACKEND]
│   ├── CustomerService.php      (Validasi & Aturan Bisnis)
│   └── api/
│       └── customers.php        (REST API Controller)
│
└── data-access/                 [LAYER 3: DATA LAYER]
    ├── Database.php             (Koneksi PDO MySQL)
    └── CustomerRepository.php   (Query SQL Parameterized)
```

Spesifikasi Endpoint REST API (JSON)

GET `/api/customers.php`

Mengambil seluruh daftar pelanggan (HTTP 200 JSON).

GET `/api/customers.php?q=kata`

Mencari pelanggan berdasarkan nama atau email.

GET `/api/customers.php?id=1`

Mengambil detail satu pelanggan untuk form Edit.

POST `/api/customers.php`

Menambah pelanggan baru + validasi (HTTP 201 / 422).

PUT `/api/customers.php?id=1`

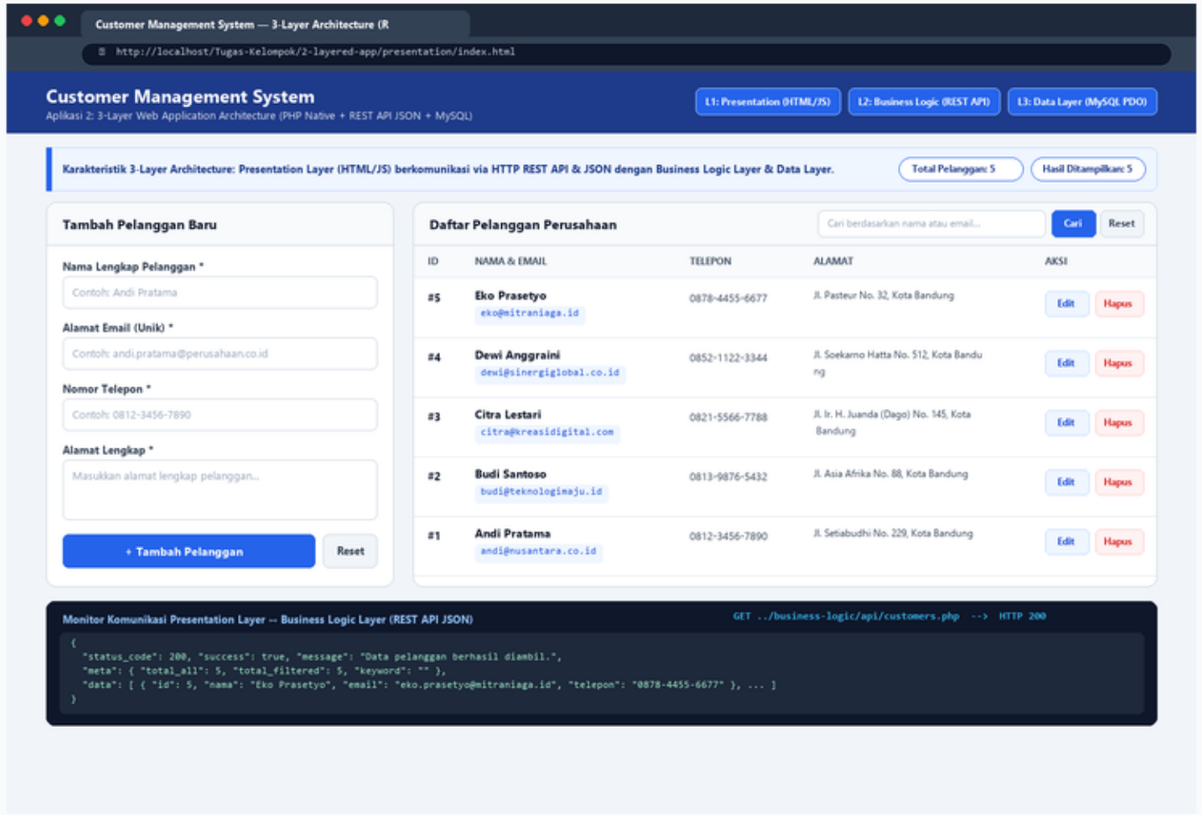
Memperbarui data pelanggan + validasi (HTTP 200 / 422).

DELETE `/api/customers.php?id=1`

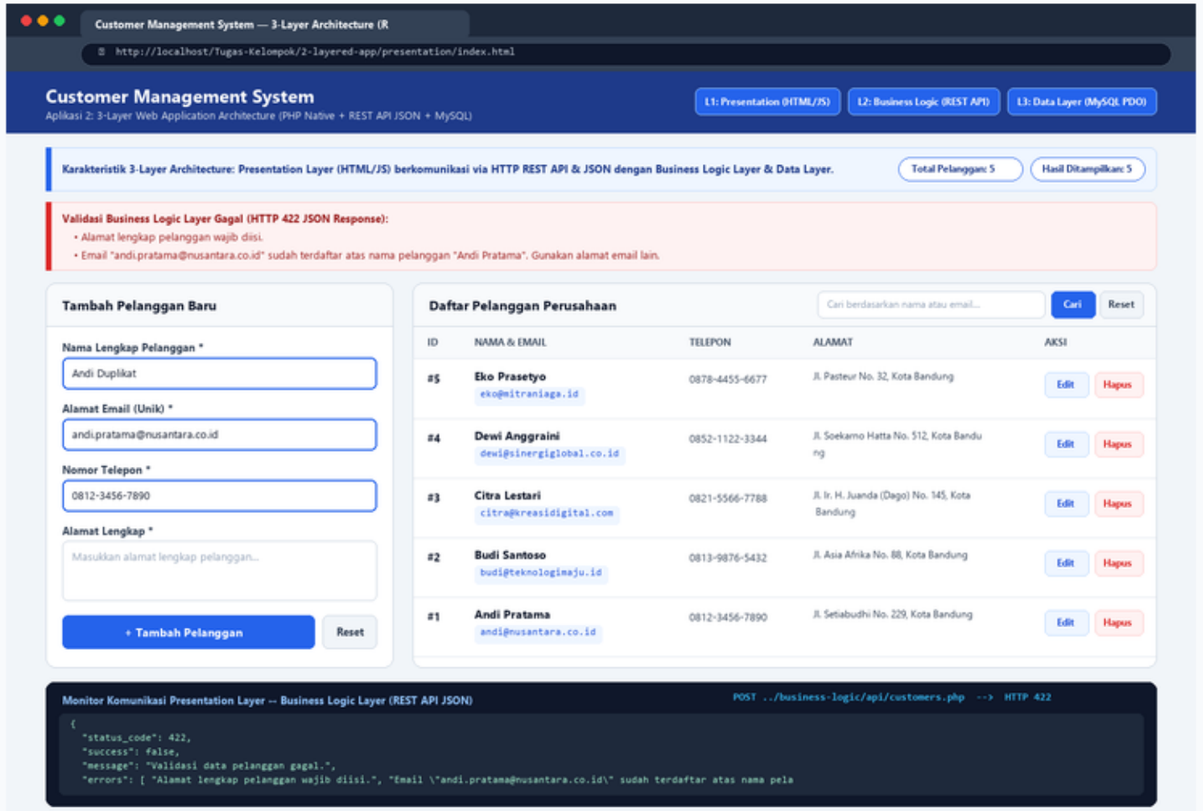
Menghapus data pelanggan dari database (HTTP 200).

Implementasi & Tampilan Aplikasi 2 (3-Layer REST API)

Tampilan Antarmuka Terpisah beserta Live Monitor Pertukaran Data REST API JSON



Dashboard 3-Layer + Monitor JSON (HTTP 200 OK)



Respons Validasi Business Logic Layer (HTTP 422 JSON)

Perbandingan Alur Proses Request & Response

Perbedaan Mekanisme Eksekusi antara Non-Layered vs 3-Layer Architecture

PERBANDINGAN ALUR EKSEKUSI: NON-LAYERED vs 3-LAYER ARCHITECTURE

Studi Kasus: Proses Tambah Pelanggan & Validasi Keunikan Email pada Customer Management System

PENDEKATAN 1: NON-LAYERED ARCHITECTURE (MONOLITIK)

Tahap 1 — Input Form HTML

Pengguna mengisi form di index.php lalu menekan tombol Submit.

↓

Tahap 2 — HTTP POST Form-Urlencoded

Browser mengirim form ke index.php (memicu pemuatan ulang halaman).

↓

Tahap 3 — Validasi Prosedural di index.php

Blok if ($_POST) memeriksa field kosong & menjalankan query cek email.

↓

Tahap 4 — Eksekusi INSERT Langsung

File index.php langsung mengeksekusi query INSERT INTO pelanggan.

↓

Tahap 5 — Query Ulang & Render HTML Utuh

File index.php menjalankan SELECT * lalu mengembalikan seluruh HTML.

PENDEKATAN 2: 3-LAYER ARCHITECTURE (REST API & JSON)

Tahap 1 — Presentation Layer (app.js)

Menangkap submit form & mengirim HTTP POST JSON via fetch() async.

↓

Tahap 2 — REST API Controller (api/customers.php)

Menerima JSON payload & meneruskannya ke CustomerService.

↓

Tahap 3 — Business Logic Layer (CustomerService.php)

Memvalidasi kelengkapan data, format email, & memanggil cek duplikasi.

↓

Tahap 4 — Data Layer (CustomerRepository.php)

Menjalankan Prepared Statement findByEmail() & create() ke MySQL.

↓

Tahap 5 — JSON Response (HTTP 201 / 422)

Backend mengirim JSON; Presentation Layer memperbarui tabel DOM.

Tabel Hasil Pengujian Fungsional Kedua Aplikasi

Seluruh 8 Skenario Pengujian (TC-01 s/d TC-08) Berjalan Sukses 100% pada Kedua Arsitektur

Kode	Fitur	Skenario Pengujian	Aplikasi 1 (Non-Layered)	Aplikasi 2 (3-Layer API)	Status
TC-01	**Tampil Pelanggan**	Menampilkan seluruh data pelanggan	Render tabel HTML langsung	GET /api/customers.php (200)	**PASS**
TC-02	**Tambah Pelanggan**	Input pelanggan baru valid & unik	POST form -> INSERT SQL	POST JSON -> HTTP 201 Created	**PASS**
TC-03	**Validasi Kosong**	Menolak form jika ada field kosong	Alert HTML 4 pesan error	HTTP 422 JSON (4 pesan error)	**PASS**
TC-04	**Validasi Format Email**	Menolak format email tidak valid	Ditolak filter_var di index.php	Ditolak CustomerService (422)	**PASS**
TC-05	**Validasi Email Unik**	Menolak email duplikat pelanggan lain	Query cek duplikat di index.php	findByEmail() menolak (422)	**PASS**
TC-06	**Pencarian**	Cari berdasarkan nama atau email	GET ?q=... (Reload Halaman)	GET ?q=... (Fetch JSON Async)	**PASS**
TC-07	**Edit Pelanggan**	Mengubah data pelanggan terdaftar	POST form_action=update	PUT /api/customers.php?id=...	**PASS**
TC-08	**Hapus Pelanggan**	Menghapus data pelanggan	GET ?action=delete&id=...	DELETE /api/customers.php?id=...	**PASS**

Tabel Perbandingan Non-Layered vs 3-Layer Architecture

Evaluasi Struktur, Pemeliharaan, Pengembangan, Reusabilitas, dan Pengujian

Aspek Evaluasi	Tanpa 3-Layer (Non-Layered)	Dengan 3-Layer Architecture
Struktur & File Program	Sederhana, menyatu dalam satu file (index.php)	**Terpisah tegas menjadi 3 direktori komponen layer**
Presentation (Frontend)	Menyatu langsung dengan kode PHP	**Terpisah murni (HTML, CSS, JS Fetch API)**
Business Logic (Validasi)	Menyatu dengan kode pemrosesan form tampilan	**Dipisahkan di dalam CustomerService & REST API**
Database Access (Data)	Langsung dari file tampilan PHP (index.php)	**Melalui komponen data access (CustomerRepository)**
Komunikasi Data	HTTP Form Submit dengan Full Page Reload	**REST API (GET, POST, PUT, DELETE) & format JSON**
Pemeliharaan (Maintenance)	Lebih sulit ketika aplikasi semakin kompleks	**Lebih terstruktur; perubahan UI aman bagi DB**
Pengembangan (Scalability)	Cepat untuk aplikasi kecil, sulit untuk tim besar	**Lebih terorganisasi & mendukung aplikasi Mobile**
Pengujian (Testing)	Cenderung lebih sulit diuji secara terpisah	**Sangat mudah diuji per layer (Unit/API Testing)**

Jawaban Pertanyaan Diskusi & Kesimpulan

Ringkasan Analisis Perbandingan Kedua Pendekatan Arsitektur

1. Mana yang lebih mudah dibuat?

Non-Layered lebih cepat dibuat di awal untuk aplikasi kecil karena hanya terdiri atas 1 file tanpa perlu membuat kontrak API.

2. Mana yang lebih mudah diperbaiki saat terjadi bug?

3-Layer jauh lebih mudah diperbaiki karena sumber bug langsung terisolasi (apakah di UI, di CustomerService, atau di SQL Repository).

3. Mana yang lebih mudah dikembangkan saat fitur bertambah?

3-Layer jauh lebih mudah dikembangkan secara modular dan mendukung penggunaan ulang REST API JSON oleh aplikasi Mobile.

4. Apa risiko jika seluruh kode dalam satu file?

Spaghetti code, perubahan desain HTML rawan merusak query SQL, sulit dikerjakan bersama dalam tim, dan sulit diuji otomatis.

5. Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tidak selalu. Untuk prototipe kilat atau skrip kecil satu fungsi, Non-Layered efisien. Namun untuk sistem bisnis nyata, 3-Layer jauh lebih unggul.